

单片机原理及应用

1. 过程性考核占总成绩40%

过程性考核满分100分

- 考勤5%
- 课堂笔记10%
- 课堂表现25%
- 阶段性学习测验30%
- 课堂作业30%

2. 考试成绩占总成绩的60%

期末考试满分100分，闭卷笔试

单片机原理及应用实验

全过程性考核满分100分

- 考勤5%
- 课堂表现10%
- 基础知识测试15%
- 实验项目 $4 \times 15\% = 60\%$
- 实验报告10%

为什么要学习单片机原理及应用？

单片机技术

现代工业自动化

电子电气

通信及物联网

智能产品... ..

ARM嵌入式、FPGA基础

毕业要求（毕业设计作品）

这门课要学习什么内容？

基础预备知识	单片机与 51 系列概述 C 语言基础
51 单片机硬件结构与核心原理	内部核心模块 时钟与复位系统
输入 / 输出 (I/O) 端口与基础外设	并行 I/O 口 中断系统 定时器 / 计数器 通信接口
扩展外设与综合应用	常用外设接口：显示设备、传感器、执行器件 综合项目实践：需求分析→电路设计→程序编写→调试优化
编程与调试技巧	程序开发流程：代码编写→编译→生成 hex 文件→烧录到单片机 常见编译错误与解决：语法错误、寄存器未定义、堆栈溢出等。 软硬件调试：万用表、示波器检测电路电压、信号；Keil 的单步执行、断点调试，变量与寄存器状态

如何学习单片机？？？

打牢基础

NO.1

NO.2

实践优先

调试能力

NO.3

进阶技巧

NO.4

心态与习惯

NO.5

一、打牢基础：明确“软硬结合”的核心逻辑

1. 吃透硬件原理，拒绝“黑箱操作”

不要急于上手写代码，先搞懂单片机的核心结构：**CPU**如何执行指令、存储器（**ROM/RAM**）如何分工、**I/O**口为什么能控制外设、时钟和复位电路的作用等。比如：**51**单片机的**P0**口为什么需要上拉电阻？定时器的计数脉冲从哪里来？这些细节决定了你能否真正“控制”硬件，而不是复制代码。

结合数据手册（**Datasheet**）学习：每款单片机的数据手册是“圣经”，比如**STC89C52**的手册会明确标注寄存器的地址、位定义、时序参数，遇到问题先查手册，比盲目百度更高效。

2. 掌握C语言的“硬件化”用法

单片机**C**语言的核心是“操作寄存器”，重点关注与硬件相关的语法：位操作（**sbit**定义引脚）、指针操作（访问特定地址寄存器）、宏定义（简化寄存器配置）等。

二、实践优先：从“最小系统”开始，逐步扩展

1. 从“点亮一盏灯”开始，积累成就感

入门阶段，用最基础的硬件搭建“最小系统”（单片机+晶振+复位电路+电源），先实现简单功能：**LED闪烁**（熟悉I/O口输出）→ **按键控制灯亮灭**（熟悉输入检测）→ **用定时器让灯按固定频率闪烁**（理解定时器原理）。每实现一个小功能，就对应一个知识点，逐步建立信心。

使用开发板，但不要依赖“模块化代码”：开发板自带的例程可以参考，但必须逐行看懂逻辑，甚至尝试用不同方法实现（比如用软件延时和定时器分别实现闪烁，对比两者的优劣）。

2. 刻意练习“模块化编程”思维

当学到外设（如LCD、传感器）时，尝试将功能封装成函数（比如`LCD1602\_Init()`、`DS18B20\_ReadTemp()`），培养“低耦合”习惯。这样做的好处是：后续复杂项目中，你能快速复用模块，而不是面对几百行代码无从下手。

三、调试能力：学会“找错”比“写对”更重要

1. 掌握硬件调试工具的使用

万用表：测量电压（判断电源是否正常、引脚是否输出高低电平）、电阻（检测电路是否短路、虚焊）。

示波器（进阶）：观察脉冲信号（如串口波形、PWM波形），

没有示波器也能调试：用LED做“指示灯”（比如串口接收数据时亮灯），通过灯的状态反推程序执行流程。

2. 软件调试：善用“断点”和“状态打印”

在Keil等IDE中，用单步执行、断点调试观察变量变化：比如定时器中断是否触发，可在中断服务函数里加一个全局变量`count++`，通过观察`count`的值判断中断是否正常。

用串口打印调试信息：当程序逻辑复杂时（如状态机），通过串口向电脑发送变量值（如“当前温度：25°C”），跟踪程序执行路径，比单纯靠“猜”高效得多。

3. 常见问题的排查思路

程序下载后没反应？先查硬件：电源是否接对？晶振是否起振？复位电路是否正常？再查软件：hex文件是否烧录正确？寄存器初始化是否错误（比如定时器没开中断允许）？

功能不稳定？考虑时序问题（如延时不够）、硬件干扰（如按键没消抖）、资源冲突（如定时器和串口共用同一个中断优先级）。

四、进阶技巧：从“模仿”到“创造”

1. 复现经典项目，理解设计思路

从简单项目开始模仿：电子时钟（定时器+LCD）→ 温湿度报警器（传感器+蜂鸣器）→ 蓝牙遥控小车（无线通信+电机驱动）。模仿时不仅要抄代码，更要画流程图，思考“为什么用这个模块？换一种方案行不行？”

2. 主动“改需求”，让自己主动解决问题

完成一个项目后，给自己加难度：比如原来的流水灯是固定速度，改成“按键可调速”；原来的温度显示是摄氏度，改成“摄氏度/华氏度切换”。这些改动会迫使你调用更多知识点（如中断、数学运算），加深理解。

3. 关注“底层逻辑”，触类旁通*

51单片机是基础，但核心原理（中断、定时器、通信协议）在STM32、Arduino等平台中是通用的。比如：理解了UART通信的波特率设置，学I2C、SPI时就能快速掌握“时钟线+数据线”的逻辑；掌握了“寄存器配置”的思路，切换到其他单片机时，只需查手册改寄存器地址即可。

五、心态与习惯：拒绝“急功近利”，享受“折腾”的过程

接受“卡壳”是常态：单片机学习中，“写了代码没反应”“硬件接线正确却不工作”是家常便饭，这时候不要放弃，用“排除法”一步步缩小问题范围（先排除硬件，再查软件）。每解决一个bug，都是一次巨大的进步。

记录学习笔记：把调试过程、关键寄存器配置、外设时序图整理下来，比如“LCD1602的RS引脚为高电平时写数据，低电平时写命令”，后续遇到同类问题可快速查阅，形成自己的“知识库”。

多逛技术社区：在CSDN、知乎、单片机论坛看别人的项目分享和问题讨论，比如“如何用51单片机驱动OLED屏”，但不要只看结论，要追问“为什么这么做”。

学前答疑

- 1、单片机有哪些？为什么初学者学51？
- 2、学单片机学什么语言？C还是汇编？
- 3、准备工作：电脑+开发板+编程软件Keil

1、单片机有哪些？为什么初学者学51？

单片机作为嵌入式系统的核心控制单元，种类繁多，根据架构、性能和应用场景可划分为多个系列。

系列	架构/位数	代表型号	主要特点	典型应用场景
51系列	8位CISC	AT89S51, STC89C52	结构简单、成本低、资料丰富、适合入门	教学实验、小家电控制（如温控风扇）689
AVR系列	8位RISC	ATmega328P	指令精简、执行速度快（1MIPS/MHz）、驱动能力强	工业控制、消费电子（如无人机飞控）79
PIC系列	8/16位RISC	PIC16F877A	低功耗（休眠电流50nA）、抗干扰强、可靠性高	医疗设备（血糖仪）、安防系统79
STM32系列	32位ARM Cortex-M	STM32F103, F407	高性能（主频达168MHz）、外设丰富（USB/以太网）	汽车电子、物联网网关、智能硬件79
ESP系列	32位Tensilica	ESP8266, ESP32	集成WiFi/蓝牙、高性价比、适合物联网	智能家居、无线传感器节点137
MSP430系列	16位RISC	MSP430G2553	超低功耗（待机电流μA级）、适合电池供电	便携设备、传感器终端79

1、单片机有哪些？为什么初学者学51？

尽管51单片机性能较弱（主频通常 $\leq 24\text{MHz}$ ），但其在初学阶段具有不可替代的优势：

一、学习曲线平缓，降低入门门槛（1）结构简单：核心寄存器（如P0-P3端口）功能明确，无需复杂配置即可控制LED、按键等基础外设。（2）开发环境成熟：Keil C51支持汇编和C语言，调试工具完善，错误排查直观。（3）对比高阶芯片：STM32需掌握库函数/寄存器双重知识，而51直接操作寄存器，避免“拷贝代码却不懂原理”的困境。

二、教学资源丰富，社区支持强大（1）高校课程标配：国内电子类专业普遍以51单片机作为教学载体，课件、实验项目均围绕其设计。（2）开源项目海量：从点亮LED到智能小车，网上有大量成熟代码和调试经验可参考。

三、低成本试错，实践门槛低（1）开发板价格低廉。（2）外设扩展简单：数码管、红外遥控等模块价格便宜，适合练手。

四、夯实底层基础，助力进阶学习：通过手动配置定时器、中断等模块，深入理解单片机工作原理，为后续学习STM32等复杂芯片打下坚实基础810。

 初学者常见误区：认为“51过时，应直接学STM32”。实际上，缺乏底层认知时直接接触库函数开发，易导致技术虚浮，调试能力薄弱10。

总结

51单片机如同“编程界的C语言”——它不是最新、最强的，但能帮初学者构建扎实的硬件控制思维。在完成基础项目后，可逐步转向ESP32（物联网）、STM32（高性能系统）等更复杂的平台。学习路径建议：51基础 → AVR/STM32进阶 → ESP32/PIC专项突破。

2、学单片机学什么语言？C还是汇编？

💡 1. 汇编语言

优势：

- 直接操作硬件，能精确控制指令执行时间和资源占用
- 生成的机器码非常精简，适合资源极其有限的低端单片机
- 对理解单片机底层工作原理（如寄存器操作、中断机制）有帮助

劣势：

- 开发效率低，代码可读性差
- 移植性差，不同架构的单片机汇编指令不兼容
- 复杂逻辑复杂项目时维护困难

适用场景：

对时序要求极高的底层驱动开发

资源极度受限的低端单片机（如 8 位 MCU）

学习单片机底层原理时的辅助工具

💡 2. C 语言

优势：

- 兼顾开发效率和执行效率，代码可读性好
- 移植性强，同一份代码稍作修改即可在不同架构单片机上运行
- 支持函数、结构体等高级特性，适合开发复杂项目
- 目前主流单片机都提供完善的 C 语言开发环境和库支持

劣势：

- 生成的机器码比汇编稍大
- 对硬件的直接控制能力不如汇编

适用场景：

绝大多数单片机应用开发（从 8 位到 32 位 MCU）

复杂控制系统、物联网设备、嵌入式产品开发团队协作开发的项目

优先学习 C 语言：是目前单片机开发的主流选择，能应对 90% 以上的应用场景。

了解汇编基础：不需要深入精通，但理解汇编有助于调试和优化关键代码。

现在的单片机开发环境（如 Keil、IAR）都对 C 语言有极佳支持，且提供了丰富的库函数，极大降低了开发难度，所以对于绝大多数开发者来说，C 语言是更优的选择。

3、准备工作：电脑+开发板+编程软件Keil

Keil作为嵌入式开发领域的经典工具套件，针对不同微控制器架构和应用场景提供了多款产品线，同时在版本迭代、功能层级和编译器技术上也存在显著区分。

Keil包含多条独立产品线，分别适配不同处理器家族：**MDK-ARM**（用于ARM微控制器开发），**KEIL C51**（用于51单片机开发），**KEIL C166**（用于C16x/ST10系列微控制器），**KEIL C251**（用于251核微控制器）。

KEIL C51版本系列区分：Keil uVision4和uVision5是两个主要版本。uVision4发布于2009年，uVision5发布于2013年。uVision5相比uVision4有更现代化的界面，支持拖拽多窗口、代码折叠等6，并且支持更新的ARM内核如Cortex-M7/M33/M556。

💡 下载和安装：

Keil官网：<https://www.keil.com/>



先安装 MDK，后安装 C51，则出现的情况就是只剩下 C51 版本的功能：

Keil MDK 与 Keil C51 共存的方法：

https://blog.csdn.net/qq_42417071/article/details/137442226

编程软件keil安装以及软件界面熟悉

*Keil*软件安装

软件界面：菜单栏（Menu Bar）、工具栏（Toolbar）、项目窗口（Project Window）、编辑窗口（Editor Window）输出窗口（Output Window）

基本配置： Edit-Configuration
Encoding
colors&fonts

下载软件STC-ISP安装以及软件界面熟悉

STC-ISP.exe http://www.MCU-Memory.com 技术支持:13922805190(姚永平) Version:4.80

Step1/步骤1: Select MCU Type 选择单片机型号

MCU Type: STC12C5201 AP Memory Range: 0000 - 03FF

Step2/步骤2: Open File / 打开文件(文件范围内未用区域填00)

起始地址(HEX) 校验和

0 0 ☒ 打开文件前清0缓冲 打开程序文件

0 0 ☒ 打开文件前清0缓冲 打开 EEPROM 文件

Step3/步骤3: Select COM Port, Max Baud/选择串行口, 最高波特率

COM: COM1 最高波特率: 115200

请尝试提高最低波特率或使最高波特率 = 最低波特率: 1200

Step4/步骤4: 设置本框和右下方 '选项' 中的选项

下次冷启动后时钟源为: ☒ 内部RC振荡器 ☐ 外部晶体或时钟

上电复位增加额外的复位延时: ☒ YES ☐ NO

振荡器放大增益(12MHz以下可选 Low): ☒ High ☐ Low

下次冷启动P1.0/P1.1: ☒ 与下载无关 ☐ 等于0/0才可以下载程序

下次下载用户应用程序时将数据Flash区一并清0: ☐ YES ☒ NO

Step5/步骤5: Download/下载 先点击下载按钮再MCU上电复位-冷启动

Download/下载 Stop/停止 Re-Download/重复下载

☐ 每次下载前重新调入已打开在缓冲区的文件, 方便调试使用

☐ 当目标代码发生变化后自动调入文件, 并立即发送下载命令

单片机出厂时的缺省设置是“P1.0/P1.1”与下载无关, P3.0/RxD, P3.1/TxD 通过 RS-232 转换器连接到电脑的普通 RS-232 串口就可以下载/编程用户应用程序到单片机内部用户应用程序区了。

如果单片机在正常工作时 P3.0/RxD 外接的是 RS-485/

程序文件 | _no_ | 串口助手 | 工程文件 |

Program File:

地址	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
000000	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000010	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000020	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000030	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000040	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000050	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000060	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000070	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000080	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000090	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0000A0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0000B0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0000C0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0000D0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0000E0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0000F0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000100	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000110	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

选项 | 自定义下载 | 脱机下载 | 检查MCU选项 | 自动增量 | ISP DEMO

P1.2/RESET2脚低于1.33V复位, 固件>V5.7有效 ☐ YES ☒ NO

时钟频率 >12MHz, 请选择 P1.2/RESET2 用作 RESET2

启动内部看门狗后禁止改看门狗分频数 ☐ YES ☒ NO

上电时自动用有关参数启动内部看门狗 ☐ YES ☒ NO

Idle(空闲)状态时内部看门狗停止计数 ☒ YES ☐ NO

上电自动启动内部看门狗时的预分频数 256

下载成功声音提示: ☒ YES ☐ NO 重复下载间隔时间(秒) 5

成功计数0 Clear 请关注www.MCU-Memory.com网站, 及时升级